

数值算法 01

姓名: 雍崔扬

学号: 21307140051

Due: Feb. 24, 2025

Problem 1

In the lecture we briefly discussed the convergence of the Gregory–Leibniz series:

$$\frac{\pi}{4} = \arctan 1 = \sum_{k=1}^{\infty} \frac{(-1)^{k-1}}{2k-1}$$

If we compute $\pi/4$ through the partial sum

$$S_{2n} = \sum_{k=1}^{2n} \frac{(-1)^{k-1}}{2k-1}$$

the convergence is slow.

Estimate the truncation error, and propose a simple correction to S_{2n} based on your error estimate.

(optional) Estimate the truncation error again for the corrected scheme.

Solution:

我们使用积分来估计截断误差:

$$\int_n^{\infty} \left(\frac{1}{4x+1} - \frac{1}{4x+3} \right) dx < \sum_{k=2n+1}^{\infty} \frac{(-1)^{k-1}}{2k-1} = \sum_{m=n}^{\infty} \left(\frac{1}{4m+1} - \frac{1}{4m+3} \right) < \int_{n-1}^{\infty} \left(\frac{1}{4x+1} - \frac{1}{4x+3} \right) dx$$

计算积分:

$$\begin{aligned} \int_n^{\infty} \left(\frac{1}{4x+1} - \frac{1}{4x+3} \right) dx &= \frac{1}{4} (\log |4x+1| - \log |4x+3|) \Big|_{x=n}^{\infty} \\ &= -\frac{1}{4} \log \left(\frac{4n+1}{4n+3} \right) \\ &= -\frac{1}{4} \log \left(1 - \frac{2}{4n+3} \right) \quad (\text{note that } \log(1-x) = -\sum_{k=1}^{\infty} \frac{x^k}{k}) \\ &= \frac{1}{4} \sum_{k=1}^{\infty} \frac{1}{k} \cdot \left(\frac{2}{4n+3} \right)^k \end{aligned}$$

$$\int_{n-1}^{\infty} \left(\frac{1}{4x+1} - \frac{1}{4x+3} \right) dx = \frac{1}{4} \sum_{k=1}^{\infty} \frac{1}{k} \cdot \left(\frac{2}{4n-1} \right)^k$$

于是我们有:

$$\frac{1}{4} \sum_{k=1}^{\infty} \frac{1}{k} \cdot \left(\frac{2}{4n+3} \right)^k < \sum_{k=2n+1}^{\infty} \frac{(-1)^{k-1}}{2k-1} < \frac{1}{4} \sum_{k=1}^{\infty} \frac{1}{k} \cdot \left(\frac{2}{4n-1} \right)^k$$

取一阶近似: **(TA: 直接取二阶近似是不精确的)**

$$\begin{aligned}
\sum_{k=2n+1}^{\infty} \frac{(-1)^{k-1}}{2k-1} &= \frac{1}{2(4n-1)} + O(n^{-2}) \\
&= \frac{1}{2(4n+1)} + O(n^{-2}) \\
&= \frac{1}{2(4n+3)} + O(n^{-2}) \\
&= \frac{1}{8n} + O(n^{-2})
\end{aligned}$$

因此对 Taylor 近似 S_{2n} 的一个简单的修正是 $S_{2n} + \frac{1}{2(4n+1)}$

(optional) 使用一阶近似修正后的截断误差为:

$$\begin{aligned}
\sum_{k=2n+1}^{\infty} \frac{(-1)^{k-1}}{2k-1} - \frac{1}{2(4n+1)} &= \sum_{m=n}^{\infty} \left(\frac{1}{4m+1} - \frac{1}{4m+3} \right) - \frac{1}{2(4n+1)} \\
&= - \sum_{m=n}^{\infty} \left(\frac{1}{4m+3} - \frac{1}{4m+5} \right) + \frac{1}{4n+1} - \frac{1}{2(4n+1)} \\
&< - \int_n^{\infty} \left(\frac{1}{4x+3} - \frac{1}{4x+5} \right) dx + \frac{1}{2(4n+1)} \\
&= -\frac{1}{4} \sum_{k=1}^{\infty} \frac{1}{k} \cdot \left(\frac{2}{4n+5} \right)^k + \frac{1}{2(4n+1)} \\
&= -\frac{1}{2(4n+5)} - \frac{1}{2(4n+5)^2} + \frac{1}{2(4n+1)} - \frac{1}{4} \sum_{k=3}^{\infty} \frac{1}{k} \cdot \left(\frac{2}{4n+5} \right)^k \\
&= -\frac{12n+19}{2(4n+1)(4n+5)^2} + O(n^{-3})
\end{aligned}$$

$$\begin{aligned}
\sum_{k=2n+1}^{\infty} \frac{(-1)^{k-1}}{2k-1} - \frac{1}{2(4n+1)} &= \sum_{m=n}^{\infty} \left(\frac{1}{4m+1} - \frac{1}{4m+3} \right) - \frac{1}{2(4n+1)} \\
&= - \sum_{m=n}^{\infty} \left(\frac{1}{4m+3} - \frac{1}{4m+5} \right) + \frac{1}{4n+1} - \frac{1}{2(4n+1)} \\
&> - \int_{n-1}^{\infty} \left(\frac{1}{4x+3} - \frac{1}{4x+5} \right) dx + \frac{1}{2(4n+1)} \\
&= -\frac{1}{4} \sum_{k=1}^{\infty} \frac{1}{k} \cdot \left(\frac{2}{4n+1} \right)^k + \frac{1}{2(4n+1)} \\
&= -\frac{1}{2(4n+1)} - \frac{1}{2(4n+1)^2} + \frac{1}{2(4n+1)} - \frac{1}{4} \sum_{k=3}^{\infty} \frac{1}{k} \cdot \left(\frac{2}{4n+1} \right)^k \\
&= -\frac{1}{2(4n+1)^2} + O(n^{-3})
\end{aligned}$$

但是两个二阶近似的差仍是一个二阶项，不够精确:

$$\frac{12n+19}{2(4n+1)(4n+5)^2} - \frac{1}{2(4n+1)^2} = \frac{16n^2+24n-3}{(4n+1)^2(4n+5)^2}$$

(存疑) 应该如何去做呢?

- 罗一凡: 使用 $\frac{1}{8n}$ 作为一阶修正项.
- 邵老师: 可以转化为定积分, 这样工具会多一点.

例如 $\frac{1}{n} = \int_0^1 x^{n-1} dx$ 和 $\frac{1}{2n+1} = \int_0^1 x^{2n} dx$, 这表明关于 n 的有理式都可表示为定积分形式。或者考虑:

$$\lim_{N \rightarrow \infty} N^2 \int_0^1 \frac{x^{4N}}{1+x^2} dx$$

- 邵老师: (存疑)

$$\begin{aligned} \sum_{k=n}^{\infty} \frac{1}{4k^2-1} - \sum_{k=n}^{\infty} \frac{1}{4(k^2-k)} &= \sum_{k=n}^{\infty} \frac{4k-1}{(4k^2-1)(4k^2-4k)} \\ \sum_{k=n}^{\infty} \frac{1}{4k^3} &\sim \int_n^{\infty} \frac{1}{4x^3} dx = -\frac{1}{8x^2} \Big|_{x=n}^{\infty} = \frac{1}{8n^2} \end{aligned}$$

最终弄出形如以下可裂项的级数:

$$\sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)} = \frac{1}{2} \sum_{k=1}^{\infty} \left[\frac{1}{k(k+1)} - \frac{1}{(k+1)(k+2)} \right] = \frac{1}{2} \cdot \frac{1}{1 \cdot 2} = \frac{1}{4}$$

Problem 2

In principle, the sine function can be evaluated through the Taylor series expansion:

$$\sin(x) = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \quad (\forall x \in \mathbb{R})$$

Let us consider two computational schemes to evaluate the sine function.

- (a) Directly truncate the Taylor series.
Make sure that the truncation error is less than the rounding error bound for any input x .
- (b) First shift x to the interval $(-\pi/2, \pi/2]$, and then apply scheme (a).
Scheme (b) is in general more accurate.
Can you provide a theoretical analysis?
- (c) Sample at least 1000 points in $[-50, 50]$ (e.g., using the MATLAB/Octave statement `linspace(-50, 50, 1000)`)
and plot the errors (e.g., using the MATLAB/Octave function `Semilogy`).
Can you explain the result?
- 本题说明 Taylor 展开不是万能的, 例如计算 $\sin(x)$ 时需要诱导函数将自变量拖进区间 $(-\pi/2, \pi/2]$ 中。事实上, $\sin(x)$ 的近似也不是用 Taylor 展开做的, 而是用最佳一致逼近来做的。

Section (a)

不失一般性, 假设 $x > 0$

取 N 为大于 $\lceil \frac{x}{2} \rceil$ 的正整数, 则前 N 项的截断误差上界为:

$$\begin{aligned} \left| \sum_{n=N}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \right| &= \frac{x^{2N+1}}{(2N+1)!} - \left(\frac{x^{2N+3}}{(2N+3)!} - \frac{x^{2N+5}}{(2N+5)!} \right) - \left(\frac{x^{2N+7}}{(2N+7)!} - \frac{x^{2N+9}}{(2N+9)!} \right) + \cdots \\ &< \frac{x^{2N+1}}{(2N+1)!} \end{aligned}$$

假设 $x, x^3, x^5, \dots$ 是通过不断乘 x^2 计算的, 则计算 x^{2n+1} 会有 $n+1$ 层累积误差.

(注意 $x^2 = x \times x$ 自带一层舍入误差)

假设整数 $(-1)^n$ 和 $(2n+1)!$ 的计算都是精确的, 则计算 $(-1)^n \frac{x^{2n+1}}{(2n+1)!}$ 会有 $n+2$ 层累积误差.

(存疑) 因此计算 $\sum_{n=0}^{N-1} (-1)^n \frac{x^{2n+1}}{(2n+1)!}$ 会有 $[(N-1)+2]+1 = N+2$ 层累积误差.

简单起见, 不考虑相消问题, 则前 N 项的部分和的舍入误差上界为:

$$\left| \text{fl} \left(\sum_{n=0}^{N-1} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \right) - \sum_{n=0}^{N-1} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \right| \leq \gamma_{N+2} \sum_{n=0}^{N-1} \frac{x^{2n+1}}{(2n+1)!} \leq \gamma_{N+2} \sinh(x)$$

$$\text{note that } \sinh(x) := \frac{e^x - e^{-x}}{2} = \frac{1}{2} \left(\sum_{n=0}^{\infty} \frac{x^n}{n!} - \sum_{n=0}^{\infty} \frac{(-x)^n}{n!} \right) = \sum_{k=0}^{\infty} \frac{x^{2k+1}}{(2k+1)!}$$

其中 γ_n 代表 n 层累积误差.

当 N 足够大时, 截断误差会小于舍入误差.

Section (b)

根据 (a) 的结论可知, 对于一般的 x , 截断误差上界为:

$$\left| \sum_{n=N}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!} \right| < \frac{|x|^{2N+1}}{(2N+1)!}$$

若 $x \notin (-\pi/2, \pi/2]$, 则将 x (或 $-x$) 以 2π 的整数倍平移至 $(-\pi/2, \pi/2]$ 后, $|x|$ 会变小, 截断误差上界会变小, 计算相同数量的项的近似结果会更加准确.

Section (c)

自定义的 `my_sin()` 函数, 可以两种方式近似 $\sin(\cdot)$ 函数:

```
% Function to compute sine using the truncated Taylor series
function sum = my_sin(x, N, modified)
    if modified == true
        % Shift x to the interval (-pi/2, pi/2]
        k = round(x/pi); % Find the integer part of x/pi
        x = x - round(x/pi) * pi; % Shift x to [-pi, pi]

        % If x is outside the range [-pi/2, pi/2], flip the sign
        if mod(k,2) == 1
            x = -x;
        end
    end
    sum = 0; % Initialize the sum
    term = x; % First term in the series
    for n = 0:N-1
        sum = sum + term; % Add the current term to the sum
        % Update the term for the next iteration
        term = -term * x^2 / ((2*n+2)*(2*n+3)); % x^(2n+3) divided by (2n+3)!
    end
```

```
end
```

函数调用:

```
% Sampling 1000 points in the range [-10, 10]
n = 1000;
x_vals = linspace(-10, 10, n)'; % 1000 points between -10 and 10
y_direct = zeros(n, 1); % For the direct Taylor series approximation
y_modified = zeros(n, 1); % For the modified Taylor series approximation

% Specify the number of terms to use in the series approximation
N = 11;

% Calculate the approximation for each method
for i = 1:n
    y_direct(i) = my_sin(x_vals(i), N, false); % Direct Taylor series
    y_modified(i) = my_sin(x_vals(i), N, true); % Modified Taylor series (shifting)
end

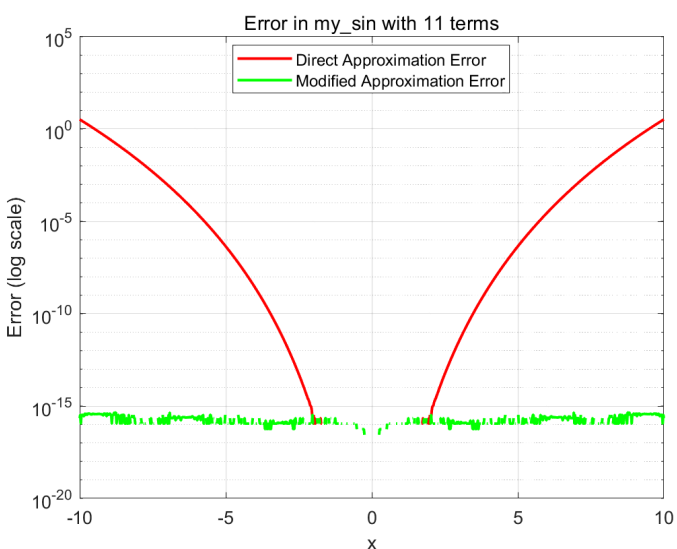
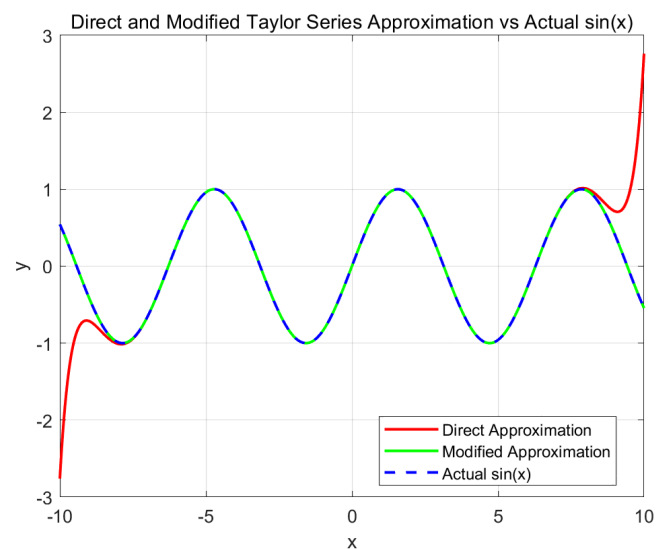
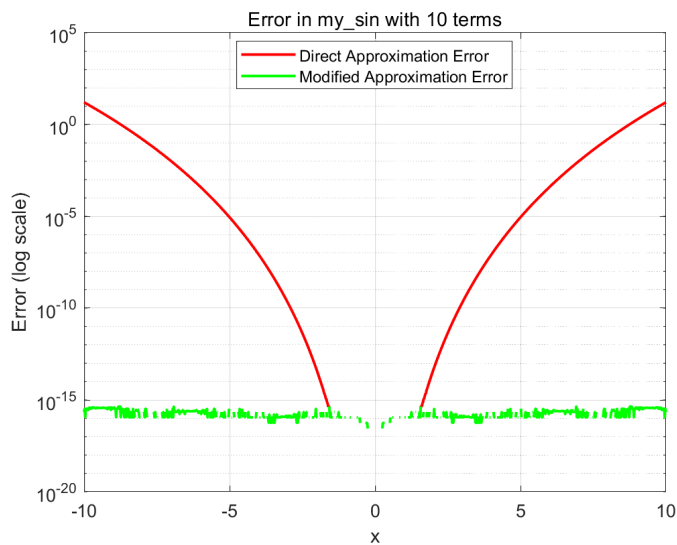
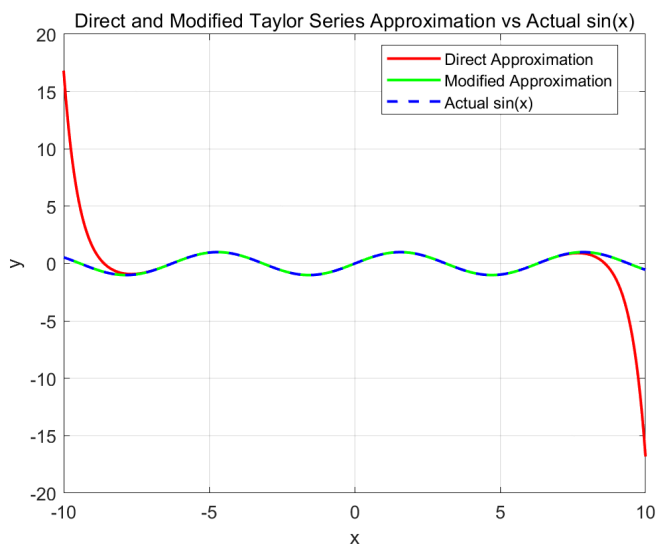
% Plot y_direct and y_modified in the same plot
figure;
plot(x_vals, y_direct, 'r', 'Linewidth', 1.5); % Direct approximation
hold on;
plot(x_vals, y_modified, 'g', 'Linewidth', 1.5); % Modified approximation
plot(x_vals, sin(x_vals), 'b--', 'Linewidth', 1.5); % Actual sine function
title('Direct and Modified Taylor Series Approximation vs Actual sin(x)');
xlabel('x');
ylabel('y');
legend('Direct Approximation', 'Modified Approximation', 'Actual sin(x)', 'Location',
'best');
grid on;

% Calculate the error between the approximation and the actual values
error_direct = abs(y_direct - sin(x_vals)); % Error for direct approximation
error_modified = abs(y_modified - sin(x_vals)); % Error for modified approximation

% Plot the error on a semilogarithmic scale
figure;
semilogy(x_vals, error_direct, 'r', 'Linewidth', 1.5); % Error for direct approximation
hold on;
semilogy(x_vals, error_modified, 'g', 'Linewidth', 1.5); % Error for modified approximation
title(['Error in my\_sin with ', num2str(N), ' terms']);
xlabel('x');
ylabel('Error (log scale)');
legend('Direct Approximation Error', 'Modified Approximation Error', 'Location', 'best');
grid on;
```

运行结果:

这验证了方法 (b) 一致优于方法 (a).



Problem 3

Using Newton's method to find the root of $\arctan(x) = 0$ is an overkill, since the unique solution, $x_* = 0$, is trivial.

However, this is a good example to see that the convergence of Newton's method relies on the initial guess. The set of real initial guesses such that Newton's method converges to x_* is of the form $(-\alpha, \alpha)$, where $\alpha > 0$. Try to calculate α with at least 10 significant decimal digits. What happens if α is used as the initial guess?

Solution:

定义 $f(x) = \arctan(x)$, 则 Newton 法的迭代格式为:

$$\begin{aligned} x_{k+1} &= x_k - \frac{f(x_k)}{f'(x_k)} \\ &= x_k - \frac{\arctan(x_k)}{1/(1+x_k^2)} \\ &= x_k - (1+x_k^2) \arctan(x_k) \end{aligned}$$

容易验证 $\alpha > 0$ 满足以下方程:

$$\alpha - (1 + \alpha^2) \arctan(\alpha) = -\alpha$$

$$\Updownarrow$$

$$\arctan(\alpha) = \frac{2\alpha}{1 + \alpha^2}$$

显然将初值 x_0 设为 α 会导致迭代序列 $\{x_k\}$ 在 $\pm\alpha$ 之间反复横跳，即是发散的。

使用二分法算得 α 的近似值为: $\alpha \approx 1.3917452002643$

Matlab 代码如下:

```
% Set the initial left and right bounds for the binary search
left = 1;           % Lower bound of the search interval
right = 2;          % Upper bound of the search interval
n = 100;            % Number of iterations for the Newton's method to check convergence

% Start the binary search loop to find an appropriate value of alpha
while (right - left) >= 1e-11
    % Compute the midpoint of the current search interval
    alpha = (left + right) / 2;

    % Call the 'test' function to check if the current alpha converges
    % within n iterations using Newton's method
    [converge, ~] = test(alpha, n);

    % If the result is true (i.e., convergence is achieved), narrow the search to the upper
    half
    if converge == true
        left = alpha; % Update the left bound to the current alpha
    else
        % If the result is false, narrow the search to the lower half
        right = alpha; % Update the right bound to the current alpha
    end
end

% After the binary search loop, 'alpha' should be an accurate estimate
% that satisfies the convergence criteria.
% Print the final value of alpha with 13 significant digits.
alpha = left;
fprintf("alpha = %.13f\n", alpha);

% Call the 'test' function again to verify the result with the final alpha
[converge, x] = test(alpha, n);

% Print whether the method converged or not based on the result
if converge
    fprintf('converge = true\n'); % Converged successfully
else
    fprintf('converge = false\n'); % Did not converge
end

% Verify the result by comparing atan(alpha) and the formula for alpha
% The verification formula is: atan(alpha) - 2*alpha / (1 + alpha^2)
```

```
fprintf("Verify: %.4e\n", atan(alpha) - 2 * alpha / (1 + alpha^2));

% Plot the convergence history of x (stored in the vector x)
% This shows how the iterates change with each step
figure;
plot(0:n, x, '-o', 'Linewidth', 2); % Plot x_k vs iteration number
xlabel('Iteration'); % Label for the x-axis
ylabel('x_k'); % Label for the y-axis
title('Convergence History of x using Newton's Method'); % Title of the plot
grid on; % Enable grid for better visualization
```

其中测试 α 的近似值作为初值是否能够收敛的函数 `test()` 定义如下:

```
% Function 'test' checks the convergence of Newton's method for a given alpha and n
iterations
function [converge, x] = test(alpha, n)
    % Initialize the vector x to store the iterates from the Newton's method
    x = zeros(n+1, 1);
    x(1) = alpha; % Set the initial value of x(1) to alpha
    converge = true; % Assume convergence unless proven otherwise

    % Iterate using Newton's method for n steps
    for i = 1:n
        % Apply the Newton's method update rule to find the next value of x
        x(i+1) = x(i) - (1 + x(i)^2) * atan(x(i));

        % Check if the change in the absolute value of x is too large (indicating non-
        convergence)
        if (abs(x(i+1)) - abs(x(i))) > eps
            converge = false; % If the change is too large, set convergence to false
            return; % Exit the loop early since convergence is not achieved
        end
    end
end
```

运行结果:

```
alpha = 1.3917452002643
converge = true
verify: -3.6070e-12
```

Problem 4

Write a program to plot the Newton fractal for the cubic equation $x^3 - 1 = 0$ over the region:

$$\{z \in \mathbb{C} : -2 \leq \operatorname{Re}(z) \leq 2 \text{ and } -2 \leq \operatorname{Im}(z) \leq 2\}$$

(optional) You are encouraged to try other polynomial equations and other regions in the complex plane.
([Mitch Richling: Newton Fractal](#))

Section (a)

定义 $f(x) = x^3 - 1$, 则 Newton 法的迭代格式为:

$$\begin{aligned}x_{k+1} &= x_k - \frac{f(x_k)}{f'(x_k)} \\&= x_k - \frac{x_k^3 - 1}{3x_k^2} \\&= \frac{2}{3}x_k + \frac{1}{3x_k^2}\end{aligned}$$

绘制 $x^3 - 1 = 0$ 的 Newton 分形的 Matlab 代码:

```
% Set up parameters
MAXITR = 128; % Maximum number of iterations
COLMAG = 10; % Color magnitude (this adjusts the color intensity over iterations)
ZROEPS = 1e-5; % Convergence threshold for Newton's method
IMGSIZ = 2048; % Image size

% Complex roots of x^3 = 1
r1 = 1;
r2 = exp(2*pi*1i/3); % r2 = -0.5 + (sqrt(3)/2)*1i;
r3 = exp(4*pi*1i/3); % r3 = -0.5 - (sqrt(3)/2)*1i;

% Create an empty image canvas
imageCanvas = zeros(IMGSIZ, IMGSIZ, 3, 'uint8'); % 3 channels for RGB image

% Set up the real and imaginary parts of the grid (canvas range [-2, 2])
[x, y] = meshgrid(linspace(-2, 2, IMGSIZ), linspace(-2, 2, IMGSIZ));

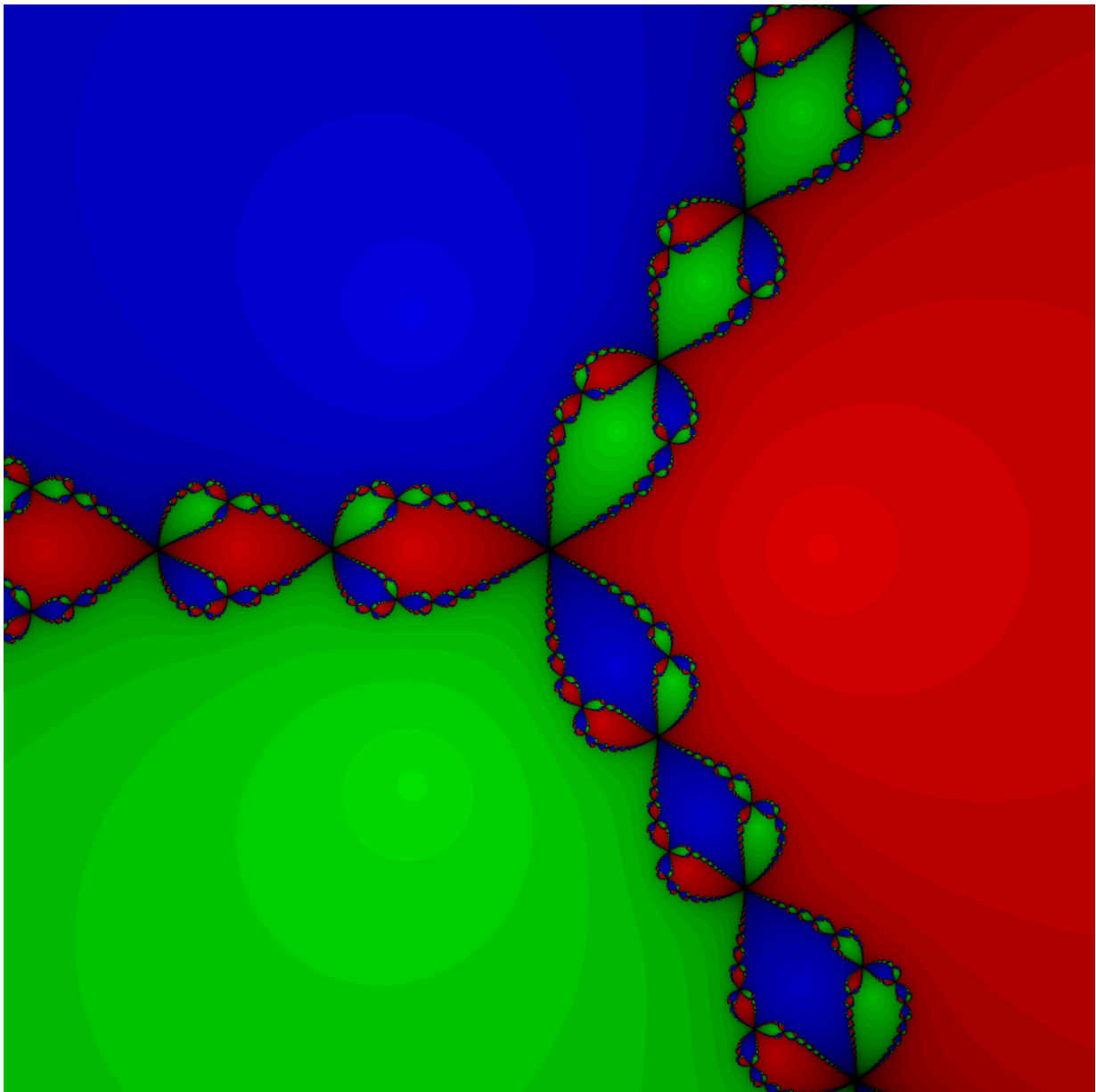
% Iterate over each pixel in the canvas
for row = 1:IMGSIZ
    for col = 1:IMGSIZ
        z = x(row, col) + 1i * y(row, col); % Convert to complex number
        for count = 1:MAXITR
            % Check convergence with roots
            if abs(z - r1) <= ZROEPS
                imageCanvas(row, col, 1) = min(255, 255 - count * COLMAG); % Red color
                break;
            elseif abs(z - r2) <= ZROEPS
                imageCanvas(row, col, 2) = min(255, 255 - count * COLMAG); % Green color
                break;
            elseif abs(z - r3) <= ZROEPS
                imageCanvas(row, col, 3) = min(255, 255 - count * COLMAG); % Blue color
                break;
            elseif abs(z) <= ZROEPS
                break; % Stop if we hit the origin
            end
            % Newton's method update (finding cube roots of unity)
            z = z - (z^3 - 1) / (3 * z^2);
        end
    end
end
end
```

```
end

% Save the image to a file
imwrite(imageCanvas, 'newton_fractal_x^3=1.png');

% Display the image
imshow(imageCanvas);
title('Convergence Image using Newton''s Method');
```

运行结果:



Section (b)

定义 $f(x) = x^6 - 1$, 则 Newton 法的迭代格式为:

$$\begin{aligned}
 x_{k+1} &= x_k - \frac{f(x_k)}{f'(x_k)} \\
 &= x_k - \frac{x_k^6 - 1}{6x_k^5} \\
 &= \frac{5}{6}x_k + \frac{1}{6x_k^2}
 \end{aligned}$$

绘制 $x^6 - 1 = 0$ 的 Newton 分形的 Matlab 代码:

```

% Set up parameters
MAXITR = 256; % Maximum number of iterations
COLMAG = 1; % Color magnitude (adjusts the color intensity over iterations)
ZROEPS = 1e-5; % Convergence threshold for Newton's method
IMGSIZ = 2048; % Image size

% Define the complex 6th roots of unity
r1 = 1;
r2 = exp(2*pi*1i/6);
r3 = exp(4*pi*1i/6);
r4 = exp(6*pi*1i/6);
r5 = exp(8*pi*1i/6);
r6 = exp(10*pi*1i/6);

% Create an empty image canvas
imageCanvas = zeros(IMGSIZ, IMGSIZ, 3, 'uint8'); % 3 channels for RGB image

% Set up the real and imaginary parts of the grid
[x, y] = meshgrid(linspace(-1, 1, IMGSIZ), linspace(-1, 1, IMGSIZ));

% Iterate over each pixel in the canvas
for row = 1:IMGSIZ
    for col = 1:IMGSIZ
        z = x(row, col) + 1i * y(row, col); % Convert to complex number
        for count = 1:MAXITR
            % Check convergence with roots
            if abs(z - r1) <= ZROEPS % Red color
                imageCanvas(row, col, 1) = min(255, 255 - count * COLMAG);
                break;
            elseif abs(z - r2) <= ZROEPS % Green color
                imageCanvas(row, col, 2) = min(255, 255 - count * COLMAG);
                break;
            elseif abs(z - r3) <= ZROEPS % Blue color
                imageCanvas(row, col, 3) = min(255, 255 - count * COLMAG);
                break;
            elseif abs(z - r4) <= ZROEPS % Yellow color
                imageCanvas(row, col, 1) = min(255, 255 - count * COLMAG);
                imageCanvas(row, col, 2) = min(255, 255 - count * COLMAG);
                break;
            elseif abs(z - r5) <= ZROEPS % Cyan color
                imageCanvas(row, col, 2) = min(255, 255 - count * COLMAG);
                imageCanvas(row, col, 3) = min(255, 255 - count * COLMAG);
                break;
        end
    end
end

```

```

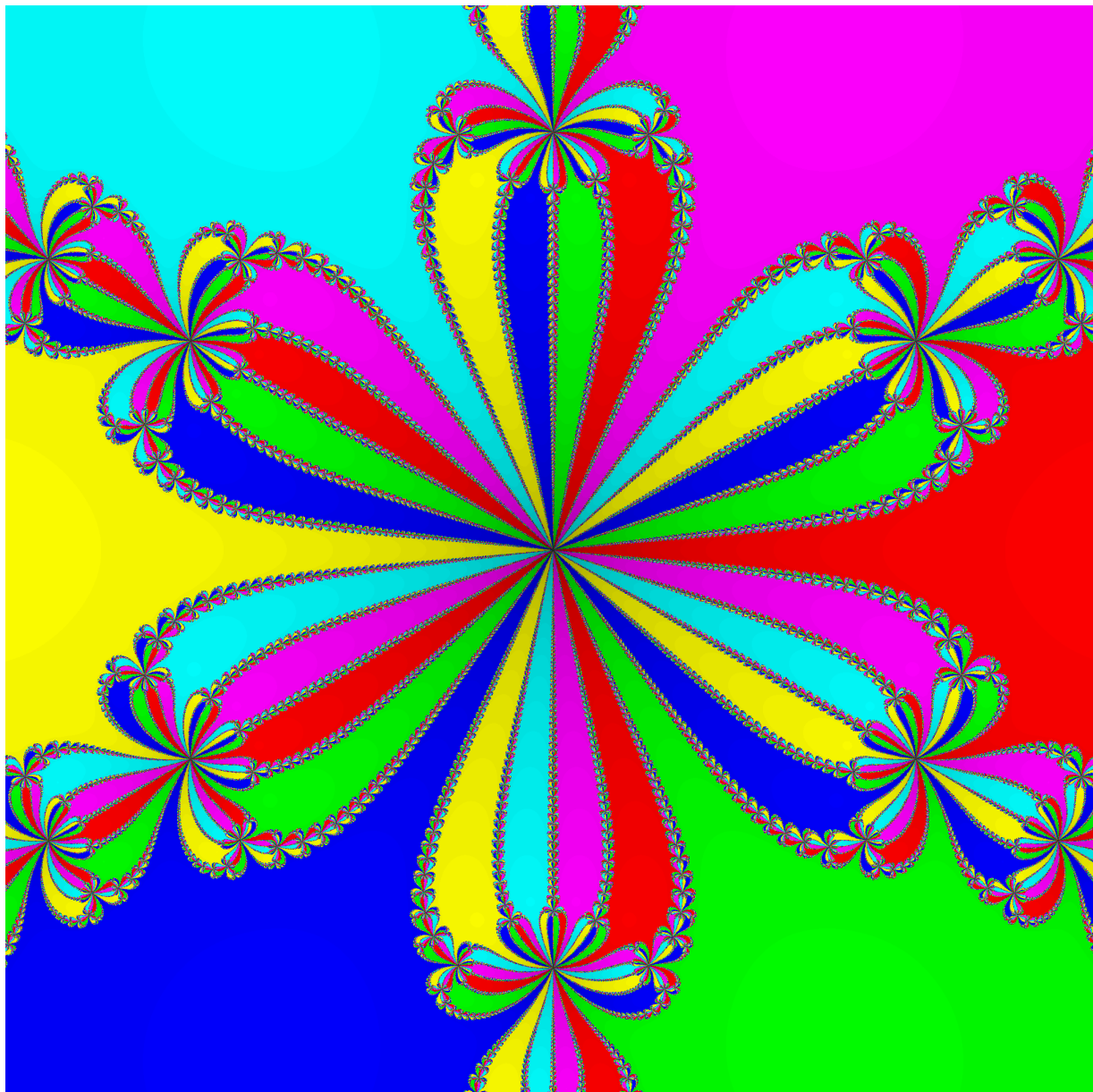
elseif abs(z - r6) <= ZROEPS    % Magenta color
    imageCanvas(row, col, 3) = min(255, 255 - count * COLMAG);
    imageCanvas(row, col, 1) = min(255, 255 - count * COLMAG);
    break;
elseif abs(z) <= ZROEPS
    break; % Stop if we hit the origin
end
% Newton's method update (finding 6th roots of unity)
z = z - (z^6 - 1) / (6 * z^5); % Apply Newton's method for  $x^6 - 1 = 0$ 
end
end
end

% Save the image to a file
imwrite(imageCanvas, 'newton_z6.png');

% Display the image
imshow(imageCanvas);
title('Convergence Image for  $x^6 - 1 = 0$  using Newton''s Method');

```

运行结果:



Problem 5 (optional)

Solve the following problem at pintia.cn and explain your algorithm.

Produce a table of the values of the series:

$$\phi(x) = \sum_{k=1}^{\infty} \frac{1}{k(k+x)}$$

for the 2001 values of x , $x = 0.000, 0.001, 0.002, \dots, 2.000$.

All entries of the table must have an absolute error less than 0.5×10^{-12} .

This problem is based on a problem from Hamming (1962),

when mainframes were very slow by today's microcomputer standards.

- **Hint:** ([zhuli19901106/zoj: Solutions to accepted problems on Zhejiang University Online Judge.](https://pintia.cn/zhuli19901106/zoj))
The problem with summing the sequence is completing the summation in the given time.
Additionally, if too many terms were to be summed,
roundoff would render any typical double precision computation useless for the desired precision.

Note that $\phi(1) = \sum_{k=1}^{\infty} \frac{1}{k(k+1)} = \sum_{k=1}^{\infty} \left(\frac{1}{k} - \frac{1}{k+1}\right) = 1$

One can then produce a series for $\phi(x) - \phi(1)$ which converges faster than the original series.

This series not only converges much faster, it also reduces roundoff loss.

This process of finding a faster converging series may be repeated to produce sequences which converge more and more rapidly than the previous ones.

The following inequality is helpful in determining how many items are required in summing the series above.

$$\sum_{k=n}^{\infty} \frac{1}{k^r} < \int_{n-1}^{\infty} \frac{1}{x^r} dx \text{ for } k \geq 1 \text{ and } r \geq 1$$

Solution:

注意到 $\phi(1) = \sum_{k=1}^{\infty} \frac{1}{k(k+1)} = \sum_{k=1}^{\infty} \left(\frac{1}{k} - \frac{1}{k+1}\right) = 1$

因此我们有:

$$\begin{aligned} \phi(x) &= \phi(x) - \phi(1) + 1 \\ &= \sum_{k=1}^{\infty} \frac{1}{k(k+x)} - \sum_{k=1}^{\infty} \frac{1}{k(k+1)} + 1 \\ &= (1-x) \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+x)} + 1 \end{aligned}$$

注意到:

$$\sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)} = \frac{1}{2} \sum_{k=1}^{\infty} \left[\frac{1}{k(k+1)} - \frac{1}{(k+1)(k+2)} \right] = \frac{1}{2} \cdot \frac{1}{1 \cdot 2} = \frac{1}{4}$$

因此我们有:

$$\begin{aligned} \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+x)} &= \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+x)} - \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)} + \frac{1}{4} \\ &= (2-x) \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+x)} + \frac{1}{4} \end{aligned}$$

注意到:

$$\sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+3)} = \frac{1}{3} \sum_{k=1}^{\infty} \left[\frac{1}{k(k+1)(k+2)} - \frac{1}{(k+1)(k+2)(k+3)} \right] = \frac{1}{3} \cdot \frac{1}{1 \cdot 2 \cdot 3} = \frac{1}{18}$$

因此我们有:

$$\begin{aligned} \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+x)} &= \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+x)} - \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+3)} + \frac{1}{18} \\ &= (3-x) \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+3)(k+x)} + \frac{1}{18} \end{aligned}$$

事实上, 上述过程可以一直进行下去, 但我们就到此为止吧.

于是我们有:

$$\begin{aligned}
\phi(x) &= (1-x) \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+x)} + 1 \\
&= (1-x) \left((2-x) \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+x)} + \frac{1}{4} \right) + 1 \\
&= (1-x) \left((2-x) \left((3-x) \sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+3)(k+x)} + \frac{1}{18} \right) + \frac{1}{4} \right) + 1
\end{aligned}$$

这样问题就转化为计算 $\sum_{k=1}^{\infty} \frac{1}{k(k+1)(k+2)(k+3)(k+x)}$

考虑其前 n 项的截断误差:

$$\begin{aligned}
\sum_{k=n+1}^{\infty} \frac{1}{k(k+1)(k+2)(k+3)(k+x)} &< \sum_{k=n+1}^{\infty} \frac{1}{k^5} \\
&< \int_n^{\infty} \frac{1}{x^5} dx \\
&= \left\{ -\frac{1}{4x^4} \right\} \Big|_{x=n}^{\infty} \\
&= \frac{1}{4n^4}
\end{aligned}$$

若多放两位, 即欲令其截断误差上界 $\frac{1}{4n^4} < 0.5 \times 10^{-14}$, 则有 $n > \sqrt[4]{5 \times 10^{13}}$

可取 $n = 3000$

于是我们有:

$$\phi(x) \approx (1-x) \left((2-x) \left((3-x) \sum_{k=1}^{3000} \frac{1}{k(k+1)(k+2)(k+3)(k+x)} + \frac{1}{18} \right) + \frac{1}{4} \right) + 1$$

上述近似可以保证绝对误差小于 0.5×10^{-12} .

算法的 C 代码为:

```

#include <math.h>
#include <stdio.h>

const double lowerbound = 0.000;
const double upperbound = 2.000;
const double inc = 0.001;

int main()
{
    int k;
    double phi_x;
    double x;

    FILE *fp = fopen("out.txt", "w");

    for(x = lowerbound; x <= upperbound; x = x + inc)
    {
        phi_x = 0;
        for(k = 1; k <= 3000; k++)
        {
            phi_x = phi_x + 1.0 / k / (k + 1) / (k + 2) / (k + 3) / (k + x);
        }
    }
}

```

```

    }
    phi_x = ((phi_x * (3 - x) + 1.0 / 18.0) * (2 - x) + 1.0 / 4.0) * (1 - x) + 1.0;
    printf("%5.3f %16.12f\n", x, phi_x);
}

return 0;
}

```

运行结果:

提交结果

题目	用户	提交时间
1007	Snivellus	2025/02/19 20:57:27
编译器	内存	用时
C (gcc)	568 / 32768 KB	41 / 10000 ms
状态 ?	评测时间	
答案正确	2025/02/19 20:57:28	

提交代码

[复制内容](#)

```

1  #include <math.h>
2  #include <stdio.h>
3
4  const double lowerbound = 0.000;
5  const double upperbound = 2.000;
6  const double inc = 0.001;
7
8  int main()
9  {
10     int k;
11     double phi_x;
12     double x;

```

Problem 6 (optional)

Design an algorithm to compute the reciprocal of a nonzero real number (i.e., $\alpha \mapsto \alpha^{-1}$) without any division operation.

([Algorithm to iteratively compute the reciprocal of a number - Stack Overflow](#))

Solution:

令 $f(x) = \frac{1}{x} - \alpha$, 则 Newton 法的迭代格式为:

$$\begin{aligned}
 x_{k+1} &= x_k - \frac{f(x_k)}{f'(x_k)} \\
 &= x_k - \frac{\frac{1}{x_k} - \alpha}{-\frac{1}{x_k^2}} \\
 &= 2x_k - \alpha x_k^2
 \end{aligned}$$

Matlab 代码:

```

rng(51); % Set the random seed for reproducibility
while true
    alpha = randn(1); % Generate a random alpha
    if alpha ~= 0
        break; % Ensure alpha is non-zero
    end
end

n = 100; % Number of iterations
x = zeros(n+1, 1); % Preallocate the x array for storing the iterates

% Set the initial guess based on the sign of alpha
if alpha > 0
    x(1) = 1;
else
    x(1) = -1;
end

% True reciprocal of alpha
reciprocal_alpha = 1 / alpha;

% Apply Newton's method update
for i = 1:n
    x(i+1) = 2 * x(i) - alpha * x(i)^2; % Update the iterates using the formula
    if abs(x(i+1) - reciprocal_alpha) < eps
        x = x(1:i+1);
        n = i;
        break
    end
end

% Output results
fprintf("alpha = %.6f\n", alpha);
fprintf("reciprocal_alpha = %.6f\n", x(end));
fprintf("absolute error = %.4e\n", abs(x(end-1) - reciprocal_alpha));

% Calculate the log of the absolute error
log_error = log(abs(x - reciprocal_alpha));

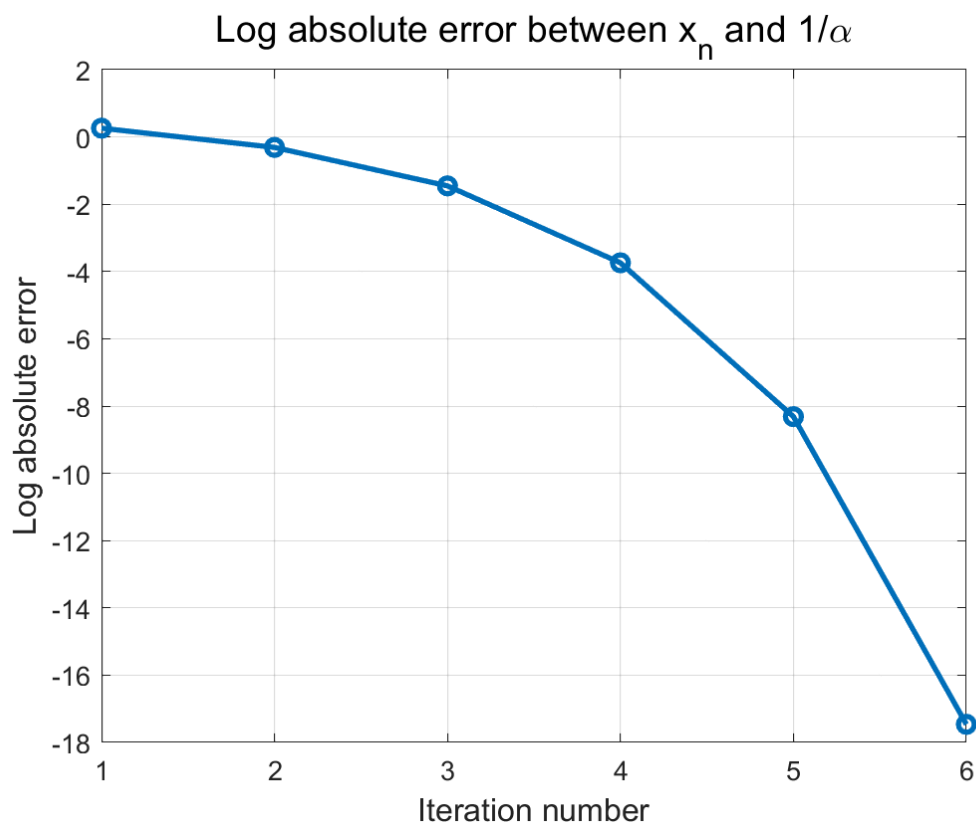
% Plot the log of the absolute error
figure;
plot(1:n+1, log_error, '-o', 'Linewidth', 2);
xlabel('Iteration number', 'FontSize', 12);
ylabel('Log absolute error', 'FontSize', 12);

```

```
title('Log absolute error between x_n and 1/\alpha', 'FontSize', 14);  
grid on;
```

运行结果:

```
alpha = 0.435427  
reciprocal_alpha = 2.296598  
absolute error = 2.6070e-08
```



Problem 7 (optional)

A backward error can sometimes be interpreted as a forward error in a certain sense.

Let us consider the following example.

Given a nonsingular matrix $A \in \mathbb{C}^{n \times n}$ and a vector $b \in \mathbb{C}^n$.

For an approximate solution $\hat{x} \in \mathbb{C}^n$ to the linear system $Ax = b$,

try to find two vector norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ such that

$$\|r\|_\alpha = \|\hat{x} - x_\star\|_\beta$$

where $r = b - A\hat{x}$ is the residual vector and $x_\star = A^{-1}b$ is the exact solution.

Solution:

定义范数 $\|\cdot\|_A$ 如下:

$$\|v\|_A := \|Av\|_2 \quad (\forall v \in \mathbb{C}^n)$$

- ① 非负性 & 正定性:

由于 A 非奇异, 故 $A^H A$ 正定.

因此 $\|v\|_A = \|Av\|_2 = \sqrt{v^H A^H A v} \geq 0$ ($\forall v \in \mathbb{C}^n$) 当且仅当 $v = 0_n$ 时取等.

- ② 齐次性:

$$\|\alpha v\|_A = \|\alpha Av\|_2 = |\alpha| \|Av\|_2 = |\alpha| \|v\|_A \quad (\forall v \in \mathbb{C}^n, \alpha \in \mathbb{C})$$

- ③ 次可加性:

$$\begin{aligned} \|u + v\|_A &= \|A(u + v)\|_2 \\ &\leq \|Au\|_2 + \|Av\|_2 \quad (\forall u, v \in \mathbb{C}^n) \\ &= \|u\|_A + \|v\|_A \end{aligned}$$

因此泛函 $\|\cdot\|_A$ 是 $\mathbb{C}^n$ 上的范数.

注意到:

$$\begin{aligned} r &= b - A\hat{x} = A(A^{-1}b - \hat{x}) = A(x_* - \hat{x}) \\ &\Downarrow \\ \|r\|_2 &= \|A(x_* - \hat{x})\|_2 = \|x_* - \hat{x}\|_A \end{aligned}$$

故我们可取 $\|\cdot\|_\alpha$ 为 $\|\cdot\|_2$, 取 $\|\cdot\|_\beta$ 为 $\|\cdot\|_A$

Problem 8 (optional)

Given a nonzero vector $x \in \mathbb{R}^n$ and a symmetric positive definite matrix $A \in \mathbb{R}^{n \times n}$, both already stored in the floating-point format.

Estimate the rounding error for evaluating $x^T A x$.

You may assume that there is no overflow or (gradual) underflow.

- **Lemma: (数值线性代数的定理 2.3.2 & 例 2.3.1)**

记 $|\cdot|$ 为逐元素取绝对值的操作, 而 $\leq$ 代表逐元素小于等于.

对于任意 $A, B \in \mathbb{C}^{n \times n}$ 我们都有:

$$\begin{cases} \text{fl}(\alpha A) = \alpha A + \Delta & (|\Delta| \leq \text{eps}|\alpha A|) \\ \text{fl}(A + B) = (A + B) + \Delta & (|\Delta| \leq \text{eps}|A + B|) \\ \text{fl}(AB) = AB + \Delta & (|\Delta| \leq \gamma_n |A| |B|) \end{cases}$$

其中 γ_n 代表 n 层浮点运算的累积相对误差的绝对值.

- **邵老师:** 另一种方法是向后误差分析, 给 A 加上扰动 ΔA

Solution:

记 $|\cdot|$ 为逐元素取绝对值的操作, 而 $\leq$ 代表逐元素小于等于.

首先计算 $y = Ax$ (记 $\hat{y} = \text{fl}(Ax)$), 根据引理我们有:

$$\hat{y} = y + \Delta y \text{ where } |\Delta y| \leq \gamma_n |A| |x|$$

其次计算 $z = x^T \hat{y}$ (记 $\hat{z} = \text{fl}(x^T \hat{y})$), 根据引理我们有:

$$\hat{z} = z + \Delta z \text{ where } |\Delta z| \leq \gamma_n |x|^T |\hat{y}|$$

于是我们有:

$$\begin{aligned}
|\text{fl}(x^\text{T}Ax) - x^\text{T}Ax| &= |\Delta z| \\
&\leq \gamma_n |x|^\text{T} |\hat{y}| \\
&\leq \gamma_n |x|^\text{T} |y| + \gamma_n |x|^\text{T} |\Delta y| \\
&\leq \gamma_n |x|^\text{T} |Ax| + \gamma_n^2 |x|^\text{T} |A| |x| \\
&\lesssim \gamma_n |x|^\text{T} |A| |x|
\end{aligned}$$

The End